

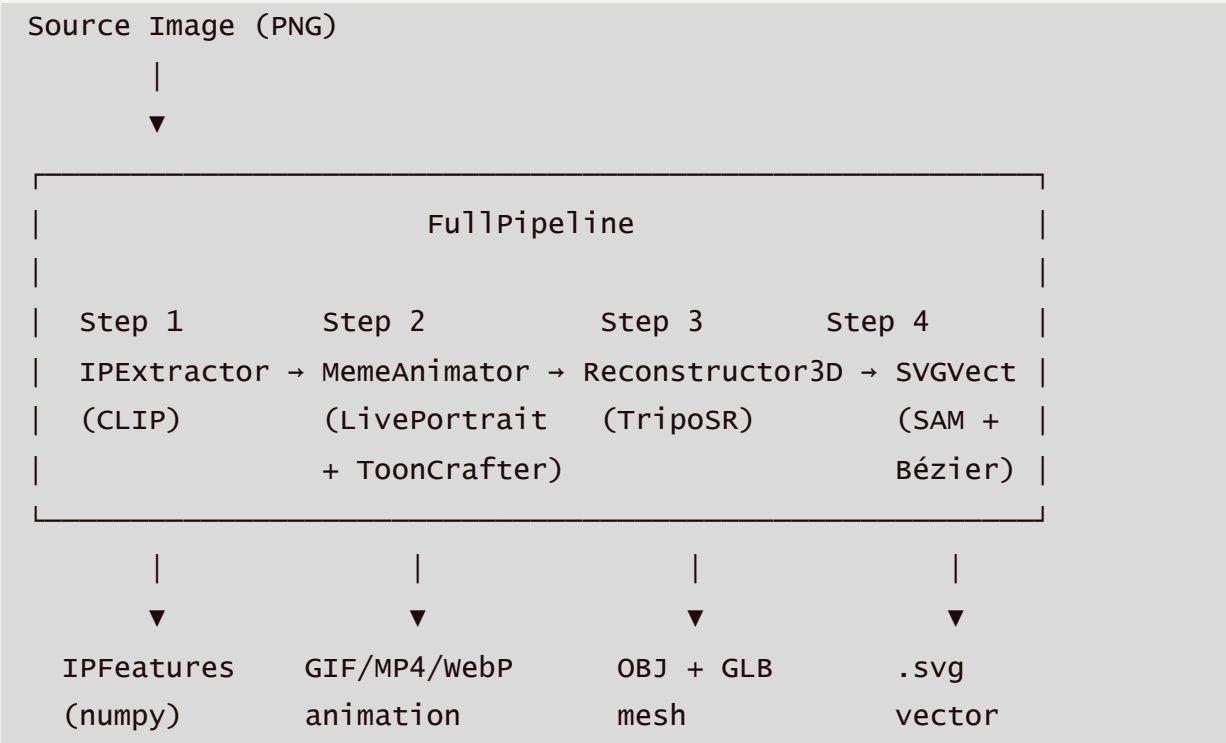
emoSVG — 架构参考（中文翻译）

本文档描述 emoSVG pipeline 的内部设计：模块边界、数据流、VRAM 管理、fallback 策略，以及关键设计决策。

目录

- 高层概览
 - 目录结构
 - 核心层
 - [ModelRegistry](#)
 - [DeviceManager](#)
 - 异常
 - 模块层
 - [IPExtractor](#)
 - [MemeAnimator](#)
 - [Reconstructor3D](#)
 - [SVGVectorizer](#)
 - Pipeline 层
 - [FullPipeline](#)
 - 局部 Pipeline
 - API 层
 - 数据流
 - VRAM 管理
 - Fallback 策略
 - 关键设计决策
-

高层概览



该 pipeline 设计为**单进程**运行。四个模块共享同一个 `ModelRegistry` 单例，由它统一执行全局 VRAM 预算和 LRU 淘汰策略。这样可以避免进程间通信的复杂性，同时让 GPU 显存使用保持可预测。

目录结构

```
emoSVG/
├─ src/
│   └─ core/                                # 共享基础设施
│       ├── model_registry.py               # 单例模型生命周期管理器
│       ├── device_manager.py               # VRAM 查询和设备迁移
│       ├── config_loader.py                # YAML 配置加载器
│       ├── exceptions.py                   # 领域异常层级
│       └─ __init__.py                       # 重新导出: ModelRegistry,
load_config
│   │
│   └─ modules/
│       ├── ip_extractor/                   # CLIP 图像编码
│       └─ meme_animator/                  # LivePortrait + ToonCrafter +
MotionDesigner
```

```

|   |   └─ reconstructor_3d/      # TriposR + mesh 后处理
|   |   └─ svg_vectorizer/        # SAM + Bézier 拟合 + SVG 组装
|   |
|   └─ pipeline/
|       └─ full_pipeline.py        # 编排全部 4 个模块
|       └─ partial_pipelines.py   # 单模块便捷封装
|       └─ base_pipeline.py       # 抽象基类
|
|   └─ api/
|       └─ main.py                 # FastAPI app 工厂
|       └─ dependencies.py         # 依赖注入 (get_pipeline)
|       └─ middleware.py           # 请求日志
|       └─ routers/                # 每个 endpoint group 一个文件
|
└─ tests/
    └─ unit/                        # 各模块单元测试
    └─ integration/                # 完整 pipeline + API endpoint 测试
    └─ smoke_test.py               # 端到端 smoke 测试
|
└─ scripts/
    └─ download_models.py          # 权重下载工具
|
└─ configs/                        # YAML 配置文件

```

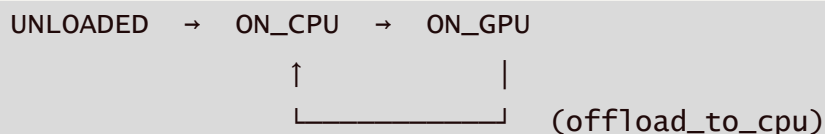
核心层

ModelRegistry

`src/core/model_registry.py`

所有模型生命周期操作的中央管理者。它是一个**进程级单例**（double-checked locking），会把每个注册模型记录为一个 `ModelEntry`。

状态



状态	含义
UNLOADED	尚未调用 loader；不占用内存
ON_CPU	权重在 RAM 中；loader 已经至少调用过一次
ON_GPU	权重在 CUDA 设备上；可以执行推理

关键方法

```
registry.register(model_id, loader_fn, estimated_vram_gb)
# 声明一个模型。惰性加载，不会立即调用 loader_fn。

registry.load_to_gpu(model_id) -> nn.Module
# 确保模型处于 ON_GPU 状态。如有需要，会触发 LRU 淘汰。

registry.offload_to_cpu(model_id)
# 将模型移回 CPU，并释放 CUDA 显存。

with registry.model_context(model_id, offload_after=True) as model:
    output = model(input)
# 上下文管理器: load -> yield -> offload。
```

LRU 淘汰

当调用 `load_to_gpu()` 且 VRAM 预算会被超出时，registry 会按最近最少使用（least-recently-used）的顺序淘汰当前驻留在 GPU 上的模型，直到有足够余量为止。每次调用 `load_to_gpu()` 时，模型的 `last_used_ts` 都会被更新。

线程安全

所有对 `_entries` 的修改都由同一个 `threading.Lock` 保护。单例创建使用单独的 `_init_lock` 和 double-checked locking。

DeviceManager

`src/core/device_manager.py`

由 `ModelRegistry` 持有。模块不会直接实例化它，而是通过 `registry.device_manager` 访问。

职责：

- 查询 VRAM 状态（`snapshot()`、`can_fit()`）
- 在设备之间移动模块（`move_to_gpu()`、`move_to_offload()`）
- 执行统一的 dtype 策略（GPU 上 float16，CPU 上 float32）
- 处理 CLIP 这类二段式模型的 tuple bundle（processor, model）

```
@dataclass
class DeviceConfig:
    device: str = "cuda"
    torch_dtype: torch.dtype = torch.float16
    vram_budget_gb: float = 10.0
    safety_margin_gb: float = 0.5    # 始终保留的空闲显存
```

有效预算为 `vram_budget_gb - safety_margin_gb`。安全余量用于防止 allocator fragmentation 导致 OOM。

异常

`src/core/exceptions.py`

领域专用异常层级：

```
EmoSVGError (base)
├─ IPExtractionError
├─ AnimationError
├─ ReconstructionError
└─ VectorizationError
```

所有模块级错误都会在继续向外传播前，被包装成对应领域异常。API 层捕获 `EmoSVGError` 并返回 HTTP 422；未预期异常返回 HTTP 500。

模块层

每个模块都遵循相同模式：

1. `__init__` 检查真实权重是否可用，并设置 `_use_fallback`
2. 如果不走 fallback，则调用 `_register_model()`，向 `ModelRegistry` 声明模型
3. 公共方法使用 `registry.model_context()` 执行推理
4. 始终提供 CPU-only fallback 路径

IPExtractor

`src/modules/ip_extractor/`

从源图像中提取角色身份特征。

主 **backend**：IP-Adapter image encoder（CLIP ViT-L/14，从 `models/ip_adapter/image_encoder/` 加载）

Fallback：从 HuggingFace 自动下载 `openai/clip-vit-large-patch14`

输出： `IPFeatures`

```
@dataclass
class IPFeatures:
    image_embeds: np.ndarray      # (1, 768) CLIP embedding
    face_embeds: np.ndarray | None # (1, 512) InsightFace
    embedding, 如可用
    preprocessed: np.ndarray      # 标准化后的图像数组
    backend_used: str              # "ip_adapter" | "clip_fallback"
```

`image_embeds` 数组会作为 `ip_image_embeds` 传给下游 `MemeAnimator`。LivePortrait 通过 `execute_portraits()` 的 `ip_adapter_embeds` kwarg, 把它作为软条件信号使用。

VRAM: 约 2.5 GB

子组件:

- `ImagePreprocessor`: 按 CLIP 输入规格 `resize` 和 `normalize`
- `IPExtractor._encode_face()`: 可选的 InsightFace 身份 `embedding`; 如果未安装 `insightface`, 会优雅跳过

MemeAnimator

`src/modules/meme_animator/`

核心交付模块。生成 Squash-and-Stretch meme 动画。

子组件:

MotionDesigner

纯数学逻辑, 不依赖模型。它会为给定 `MemeExpression` 生成一组 `SquashParams`, 描述运动曲线。

运动阶段:

```
wind-up (2 frames) → attack (3 frames) → hold (N) → bounce (M) →
settle (3 frames)
```

Wind-up 是反向预备动作（略微朝峰值的反方向运动），用于卡通动画中的 anticipation。每个阶段都在 neutral 与 peak SquashParams 之间线性插值。

预设定义在 motion_designer.py 的 _PRESETS: dict[MemeExpression, SquashParams] 中。

SquashParams

包含 11 个带校验字段的 Pydantic model:

字段	范围	效果
eye_bulge_scale	0.1–5.0	眼睛尺寸倍率
eye_squint_scale	0.1–2.0	眼睛垂直压缩
brow_raise_offset	-1.0–1.0	眉毛垂直偏移
jaw_drop_scale	0.5–3.0	下巴张开倍率
mouth_width_scale	0.5–2.5	嘴巴宽度倍率
mouth_corner_offset	-1.0–1.0	微笑 (+) /皱眉 (-)
head_squash_scale	0.3–1.5	头部垂直压缩
head_stretch_scale	0.5–2.0	头部垂直拉伸
head_tilt_deg	-30–30	头部 roll 角度
hold_frames	1–30	峰值表情保持帧数
bounce_frames	0–10	overshoot 回弹帧数

LivePortraitWrapper

把 SquashParams 转换为 LivePortrait 的 63 维表情系数空间，并渲染帧。

表情系数映射（63 维 = 21 个关键点 × 3 个坐标）:

dims	0- 5	left-eye (z → bulge, y → squint)
dims	6-11	right-eye
dims	12-17	brow (y → raise/lower)
dims	18-23	nose
dims	24-29	upper-lip
dims	30-35	lower-lip
dims	36-41	jaw/chin (y → drop)
dims	42-47	mouth-corner (x → width, y → smile/frown)
dims	48-62	remaining face keypoints

真实推理路径（当权重存在时）：

```

source_bgr
  |
  ▼
_preprocess_source()    → (1, 3, H, w) float tensor [-1, 1]
  |
  ▼
pipeline.get_kp_info()  → kp_source dict (pitch, yaw, roll, t,
exp, scale, kp)
  |
  ▼
_build_driving_info()   → x_d_info dict (kp_source + SquashParams
deltas)
  |
  ▼
pipeline.execute_portraits(img_rgb, x_s_info, x_d_info,
ip_adapter_embeds=...)
  |
  ▼
_tensor_to_bgr()        → HxWx3 uint8 BGR frame

```

`render_sequence()` 只提取一次源图像关键点，并在整个帧 batch 中复用，避免重复调用 `get_kp_info()`。

MEME_MOTION_TEMPLATES：五个硬编码的极端表情模板（`SHOCK_EXTREME`、`LAUGH_EXTREME`、`RAGE_EXTREME`、`CRY_EXTREME`、`SMUG_EXTREME`），定义为 63 维 `exp_delta` 数组。这些是 `MotionDesigner` 插值时靠近的峰值目标。

VRAM: 约 4.5 GB

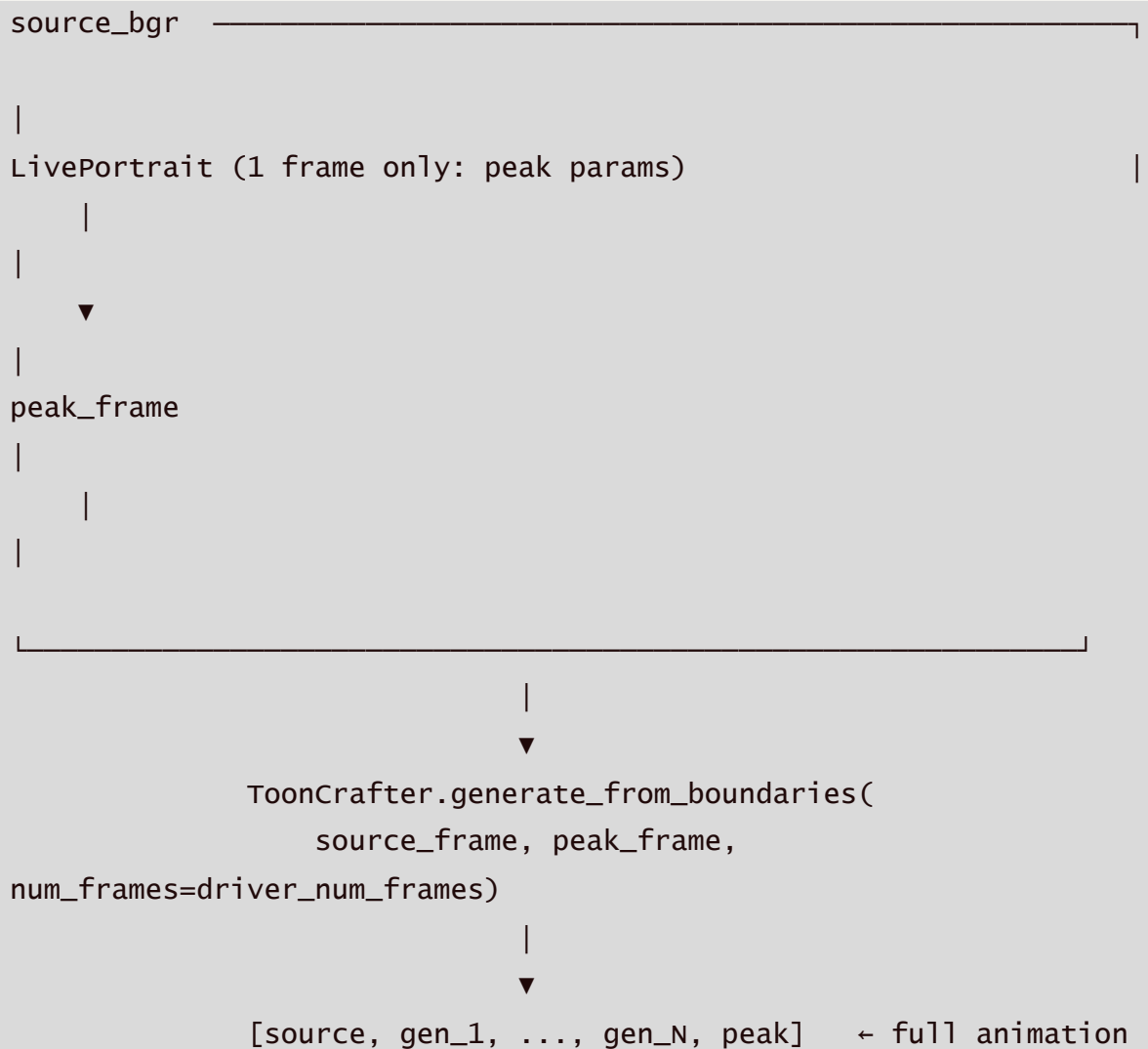
Fallback: 使用 OpenCV affine-warp (缩放 + 旋转 + 眼部区域放大)

ToonCrafterWrapper

面向卡通的视频生成组件，根据 `AnimationRequest` flags 选择两种运行模式。

模式 1 — 主驱动器 (`use_toon_crafter=True`, `use_toon_crafter_as_driver=True`):

LivePortrait 只渲染单个峰值表情帧。然后 ToonCrafter 的视频扩散 backbone 以 `(source_frame, peak_frame)` 作为边界条件生成完整动画序列，输出 `driver_num_frames` 个时间一致帧。



这种方式可以产生真正的 Squash-and-Stretch 运动，因为扩散模型是在图像空间插值，而不是扭曲关键点坐标，因此绕过了 LivePortrait 人脸表情空间的限制。

模式 2 — 插帧器 (`use_toon_crafter=True`, `use_toon_crafter_as_driver=False`):

LivePortrait 先根据 MotionDesigner 曲线渲染完整关键帧序列。ToonCrafter 再在每对连续关键帧之间插入 `frames_between` 个插值帧，使卡通运动更平滑。

```
LivePortrait keyframes: [F0, F1, F2, F3, ...]
After ToonCrafter (n=3): [F0, i1, i2, i3, F1, i4, i5, i6, F2, ...]
```

`smooth_sequence()` 会对每对连续帧调用 `interpolate(start, end, n)` 并拼接结果，同时避免重复边界帧。

VRAM: 约 8 GB

Fallback:

- Driver mode: 从 source 到 peak 的 cosine-schedule pixel blend
- Interpolator mode: 帧之间的 linear pixel blend

AnimationRequest flags 汇总

FLAG	默认值	效果
<code>use_toon_crafter</code>	<code>False</code>	启用 ToonCrafter (任一模式)
<code>use_toon_crafter_as_driver</code>	<code>False</code>	使用 driver mode (需要 <code>use_toon_crafter=True</code>)
<code>driver_num_frames</code>	<code>16</code>	driver-mode 输出总帧数 (2–64)
<code>frames_between</code>	<code>4</code>	interpolator mode 每个间隔插入的帧数 (1–16)

Reconstructor3D

src/modules/reconstructor_3d/

通过 TripoSR 进行单图 3D 重建。

Pipeline:

```
source image
  |
  ▼
_load_image()
  ├── optional rembg background removal
  └── _recenter_foreground() ← 裁剪 tight bbox, 并 pad 为正方形,
                              让主体填充 foreground_ratio (0.85)
  ▼
TripoSR inference
  └── scene_codes → extract_mesh(mc_resolution)
  |
  ▼
MeshProcessor.process()
  ├── remove degenerate faces (trimesh 4.x: nondegenerate_faces()
mask)
  ├── merge duplicate vertices
  └── compute vertex normals
  |
  ▼
MeshExporter.export()
  └── OBJ + GLB files
```

前景重居中: 当主体填充输入帧约 85% 时, TripoSR 效果最好。

`_recenter_foreground()` 会检测 alpha 通道 bounding box, 计算最小包围正方形, 并 pad 到一个满足 `side / foreground_ratio = canvas_side` 的画布。

VRAM: 约 6 GB

Fallback: trimesh UV-sphere (32×32 subdivisions)

SVGVectorizer

`src/modules/svg_vectorizer/`

把栅格图像转换为分层 SVG。

Pipeline:

```
source image
  |
  ▼
Segmentor.segment()
  ├── SAM: automatic mask generation (all regions)
  └── fallback: OpenCV K-means colour clustering + contour
      extraction
        |
        ▼
BezierFitter.mask_to_paths() (per mask)
  └── schneider algorithm: contour points → cubic Bézier curves
        |
        ▼
SVGBuilder.build()
  └── one <g> layer per mask, filled with mean region colour
```

输出： 分层 SVG，每个语义区域对应一个独立 `<g>` 元素，并带有唯一 `id`。图层命名遵循 `layer_naming` 策略，默认基于颜色命名。

VRAM： 约 2.4 GB (SAM ViT-H)

Fallback： OpenCV K-means (k=8 colors) + `findContours` → polyline paths

Pipeline 层

FullPipeline

src/pipeline/full_pipeline.py

按顺序编排全部四个模块。它持有模块实例，并将共享的 `ModelRegistry` 传给每个模块。

```
class FullPipelineRequest:
    source_image_path: Path
    expression: MemeExpression
    output_format: str          # gif | mp4 | webp
    fps: int
    resolution: tuple[int, int]
    run_3d: bool
    run_svg: bool
    use_source_for_3d_svg: bool # False = 使用峰值关键帧; True = 使用
源图
    use_toon_crafter: bool
    frames_between: int
```

二级输入解析：第 3 步和第 4 步共享同一个二级输入路径，该路径只计算一次：

- `use_source_for_3d_svg=False`（默认）：把动画峰值关键帧（keyframe list 的中间帧）保存到 `outputs/_tmp_keyframes/`，并同时作为 TripoSR 与 SAM 的输入
- `use_source_for_3d_svg=True`：直接使用原始源图像

这样在同时启用 `run_3d` 和 `run_svg` 时，可以避免把关键帧写入磁盘两次。

局部 Pipeline

src/pipeline/partial_pipelines.py

API 中用于单模块调用的便捷封装：

CLASS	ENDPOINT	MODULE
<code>AnimatePipeline</code>	<code>POST /animate</code>	<code>MemeAnimator</code>

CLASS	ENDPOINT	MODULE
ReconstructPipeline	POST /reconstruct	Reconstructor3D
VectorizePipeline	POST /vectorize	SVGVectorizer

每个类都有一个 `from_config()` classmethod，并将构造逻辑委托给底层模块的 `from_config()`。

API 层

`src/api/`

FastAPI 应用包含五组 router：

ROUTER	PREFIX	DESCRIPTION
<code>generate</code>	<code>/generate</code>	完整 pipeline
<code>animate</code>	<code>/animate</code>	仅动画
<code>reconstruct</code>	<code>/reconstruct</code>	仅 3D
<code>vectorize</code>	<code>/vectorize</code>	仅 SVG
<code>batch</code>	<code>/batch</code>	多图 batch

另外还有两个工具 endpoint： `GET /health`、 `GET /expressions`、 `GET /status`。

依赖注入： `dependencies.py` 中的 `get_pipeline()` 返回 `FullPipeline` 单例。测试通过 `app.dependency_overrides` 覆盖该依赖。

输入校验： 所有数值参数都会在调用 pipeline 前由 router handler 校验（width 64–2048, height 64–2048, fps 8–60, frames_between 1–16, output_format 属于 {gif, mp4, webp}）。

Batch endpoint： 按顺序处理图像（单进程约束）。单张图失败会被捕获并写入结果，不会中断剩余图像。硬上限：每个请求最多 8 张图（`_MAX_BATCH = 8`）。

Workers： 始终为 `workers=1`。 `ModelRegistry` 单例不是 fork-safe。

数据流

```
POST /generate
|
|— save upload → tmp file
|
|— FullPipeline.execute(FullPipelineRequest)
|   |
|   |— IPExtractor.extract()
|   |   |— CLIP encode → IPFeatures(image_embeds)
|   |
|   |— MemeAnimator.generate()
|   |   |— MotionDesigner.design(expression) →
[SquashParams]
|   |   |— LivePortraitWrapper.render_sequence(
|   |   |   source_bgr, param_sequence,
ip_image_embeds)
|   |   |   |— [rendered frames]
|   |   |— ToonCrafterWrapper.smooth_sequence()
[optional]
|   |   |— FrameComposer.compose() → GIF/MP4/WebP file
|   |
|   |— Reconstructor3D.reconstruct() [if run_3d]
|   |   |— _recenter_foreground()
|   |   |— TriposSR inference
|   |   |— MeshProcessor.process()
|   |   |— MeshExporter.export() → OBJ + GLB
|   |
|   |— SVGVectorizer.vectorize() [if run_svg]
|   |   |— Segmentor.segment() → masks
|   |   |— BezierFitter.mask_to_paths() per mask
|   |   |— SVGBuilder.build() → .svg file
|
|— delete tmp file
|
|— GenerateResponse(animation_path, frame_count, ...)
```

VRAM 管理

pipeline 的理论总 VRAM 需求约为 15.4 GB（所有模型同时加载）。实际运行中，模型会一次只加载一个，并在步骤之间被淘汰。

MODEL	VRAM ESTIMATE
CLIP (IPExtractor)	2.5 GB
LivePortrait	4.5 GB
ToonCrafter	8.0 GB
TripoSR	6.0 GB
SAM ViT-H	2.4 GB

在 10 GB GPU 上执行完整 pipeline 时的淘汰顺序：

- 1. CLIP 加载（2.5 GB）→ 提取特征 → offload
- 2. LivePortrait 加载（4.5 GB）→ 渲染全部帧 → offload
- 3. ToonCrafter 加载（8.0 GB）→ 平滑序列 → offload
- 4. TripoSR 加载（6.0 GB）→ 重建 mesh → offload
- 5. SAM 加载（2.4 GB）→ 分割图像 → offload

因为每次只有一个模型在 GPU 上，所以每一步都能放进 10 GB 预算。`model_context()` 中默认的 `offload_after=True` 会保证自动清理。

如果 `use_toon_crafter=False`（默认），ToonCrafter 永远不会加载，峰值 VRAM 降为 6 GB。

Fallback 策略

每个模块都会在构造时检测权重是否可用，并设置 `_use_fallback: bool`。fallback 路径始终是 CPU-only 实现，会产出结构正确但视觉上更简化的结果。

MODULE	REAL BACKEND	FALLBACK
IPExtractor	IP-Adapter CLIP ViT-L/14	HF CLIP ViT-L/14（自动下载）
LivePortraitWrapper	LivePortrait pipeline	OpenCV affine warp

MODULE	REAL BACKEND	FALLBACK
ToonCrafterWrapper (driver)	ToonCrafter generate_from_boundaries	Cosine-schedule pixel blend
ToonCrafterWrapper (interpolator)	ToonCrafter interpolate	Linear pixel blend
Reconstructor3D	TripoSR	trimesh UV-sphere
SVGVectorizer	SAM ViT-H	OpenCV K-means contours

这一设计意味着完整测试套件（221 tests）无需 GPU 或已下载权重也能运行。

关键设计决策

ToonCrafter 双模式设计

ToonCrafter 在请求时可以承担两种角色。在 driver mode（`use_toon_crafter_as_driver=True`）中，LivePortrait 只渲染峰值帧，ToonCrafter 通过它的视频扩散 backbone 负责完整时间序列生成。这种方式是在图像空间插值，而不是扭曲关键点坐标，因此可以产生真正的卡通 Squash-and-Stretch。在 interpolator mode 中，它填补完整 LivePortrait 关键帧序列之间的空隙。两种模式共享同一个 `ToonCrafterWrapper` 和 `ModelRegistry` entry，只是 `MemeAnimator.generate()` 中的调用路径不同。

LivePortrait 表情空间限制

LivePortrait 的 63 维表情系数空间是在人脸数据上校准的。为了实现真正 meme 级夸张效果而把系数推到人体生理极限之外时，会产生 mesh artifacts。Driver mode 通过 ToonCrafter 的扩散 backbone 进行时间生成来绕过这个问题，LivePortrait 只负责渲染单个峰值帧作为视觉目标。

单进程，共享 registry

多进程 worker 会要求每个进程加载自己的模型副本，从而成倍增加 VRAM 使用。单进程且 `workers=1` 可以让所有模块共享一个 registry 和同一份 VRAM 预算。

LivePortrait bundle 注册

LivePortrait 有 5 个子网络（AppearanceExtractor、MotionExtractor、WarpingNetwork、SPADEGenerator、StitchingRetargeting），它们之间存在复杂的张量流。把整个 `LivePortraitPipeline` 对象作为单一 bundle 注册，可以避免手动管理中间张量，并与官方 API 设计保持一致。

源关键点每个序列只提取一次

`render_sequence()` 只对源图像调用一次 `get_kp_info()`，并将结果复用于所有帧。这样可以避免通过 `MotionExtractor` 执行 N 次冗余 forward pass。

峰值关键帧作为 3D/SVG 输入

默认情况下，`TripSR` 和 `SAM` 接收动画峰值关键帧（最夸张的 pose），而不是原始源图。这会让 3D mesh 和 SVG vector 与动画角色的表情匹配。设置 `use_source_for_3d_svg=True` 可以改用 canonical source image。

二级输入只计算一次

当 `run_3d=True` 且 `run_svg=True` 时，峰值关键帧只会被保存到磁盘一次，并同时被两个步骤复用。早期版本会调用 `_resolve_secondary_input()` 两次，导致文件被写入两次。

trimesh 4.x API

`trimesh 4.x` 移除了 `remove_degenerate_faces()`。mesh processor 改用 `nondegenerate_faces()` boolean mask：
`mesh.update_faces(mesh.nondegenerate_faces())`。

DeviceManager 处理 tuple bundles

`CLIP` 会被加载为 `(processor, model)` tuple。`move_to_gpu()` 和 `move_to_offload()` 会检测 tuple，并分别移动其中每个 `nn.Module` 元素，同时保持容器类型不变。

torch 的 TYPE_CHECKING guard

`live_portrait.py` 在类型注解中使用 `torch.Tensor`，但为了避免 fallback mode 下的 import 开销，会在方法内部惰性导入 `torch`。`TYPE_CHECKING` guard 让类型检查器能看到这些注解，同时不产生运行时 import。